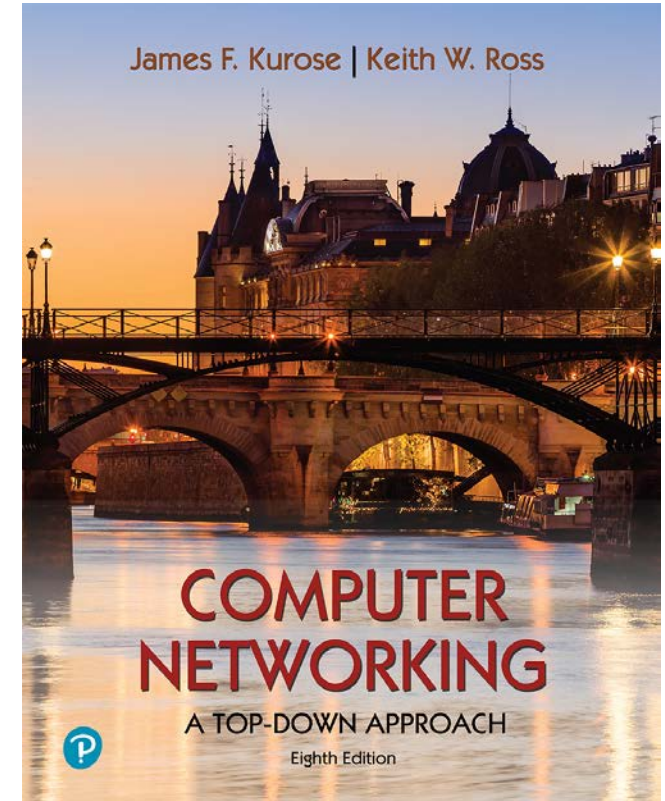


Chapter 2

Application Layer

All material copyright 1996-2023
J.F Kurose and K.W. Ross, All Rights Reserved



Computer Networking: A Top-Down Approach

8th edition

Jim Kurose, Keith Ross
Pearson, 2020

Application Layer: Overview

- Principles of network applications
- Web and HTTP
- E-mail, SMTP, IMAP
- The Domain Name System DNS
- P2P applications
- video streaming and content distribution networks
- **socket programming with UDP and TCP**

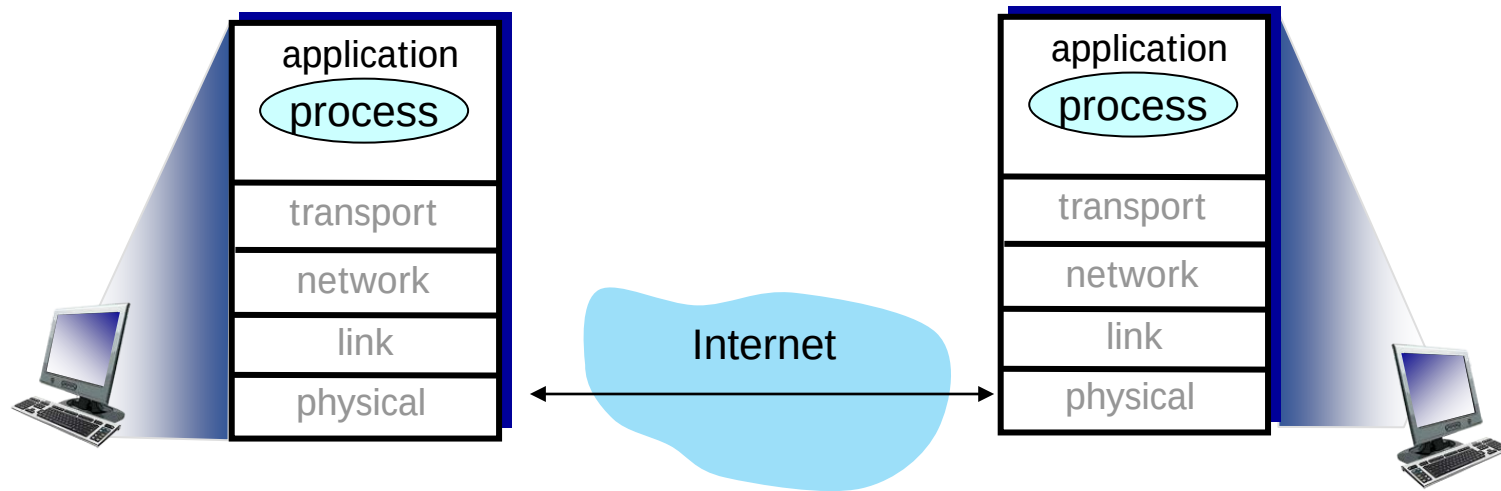


Socket programming

- Socket abstraction
- UDP socket
- TCP sockets

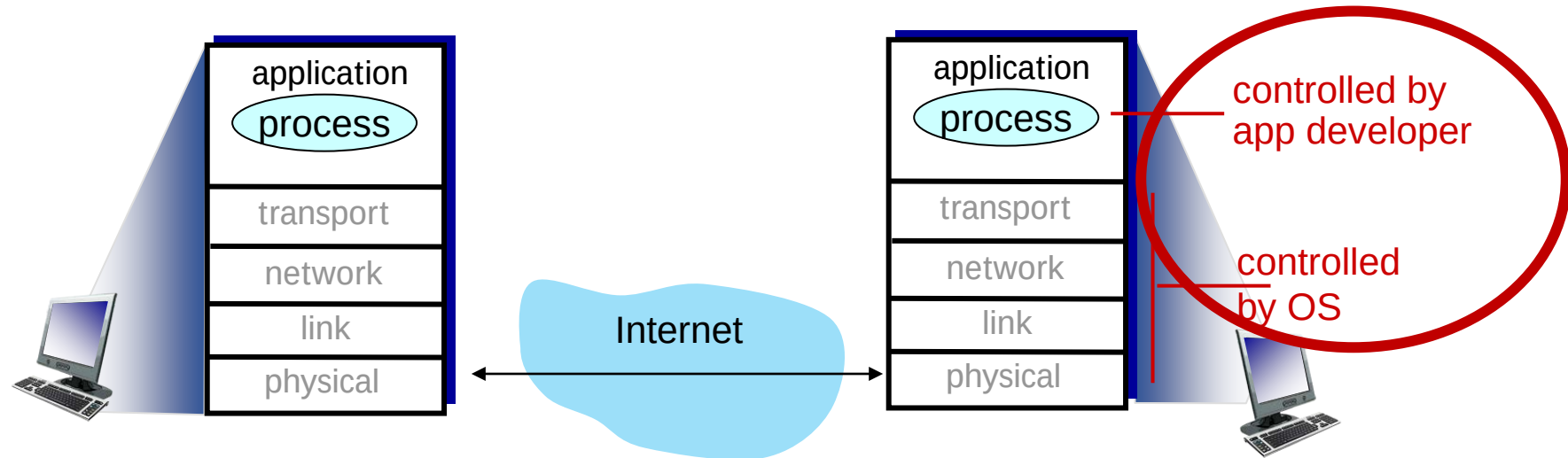
Socket programming

goal: learn how to build client/server applications that communicate using sockets



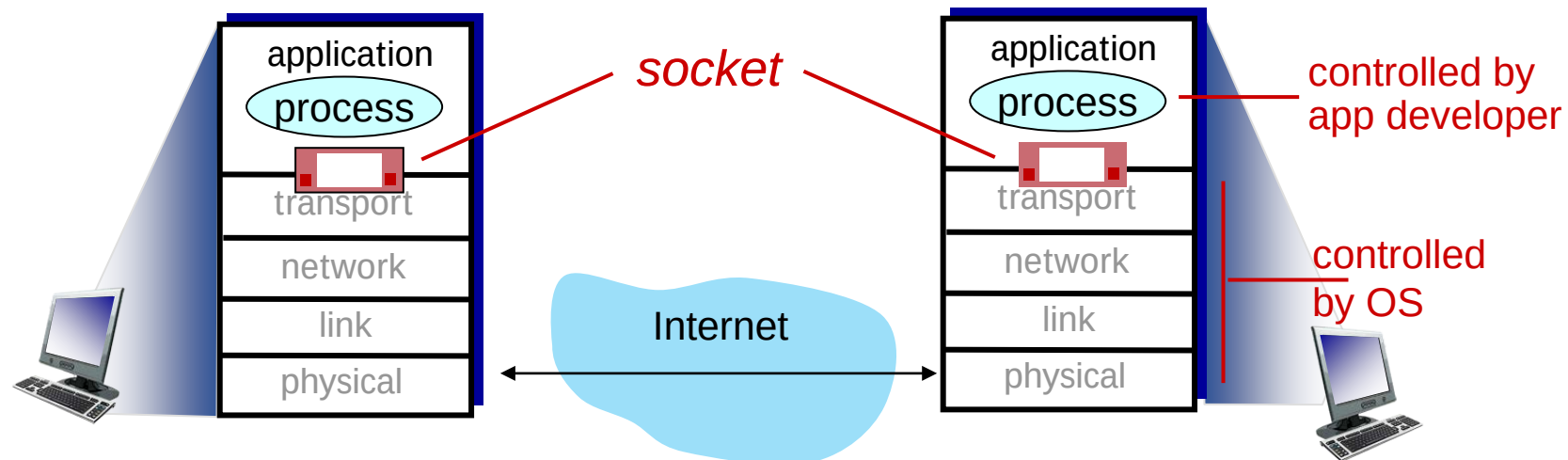
Socket programming

From an operating system point of view, your application is written in user space.



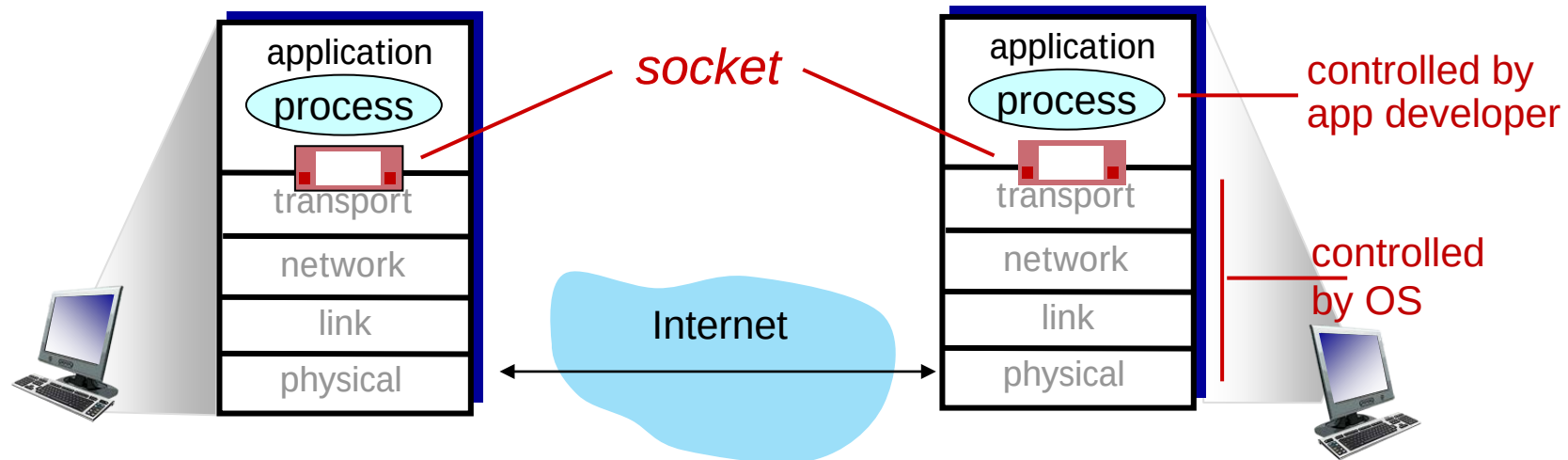
Socket programming

- process sends/receives messages to/from its **socket**
- socket analogous to door
 - sending process shoves message out door



Socket programming

- process sends/receives messages to/from its **socket**
- socket analogous to door
 - sending process shoves message out door
 - sending process relies on transport infrastructure on other side of door to deliver message to socket at receiving process
 - two sockets involved: one on each side



Socket programming

Two socket types for two transport services:

- *TCP*: reliable, congestion controlled, flow controlled, byte stream-oriented
- *UDP*: unreliable datagram

Socket programming

Two socket types for two transport services:

- *UDP*: unreliable datagram
- *TCP*: reliable, byte stream-oriented

Application Example:

1. client reads a line of characters (data) from its keyboard and sends data to server
2. server receives the data and converts characters to uppercase
3. server sends modified data to client
4. client receives modified data and displays line on its screen

Socket programming with UDP

UDP: no “connection” between client and server:

- no handshaking before sending data
- sender explicitly attaches IP destination address and port # to each packet
- receiver extracts sender IP address and port# from received packet

Socket programming with UDP

UDP: no “connection” between client and server:

- no handshaking before sending data
- sender explicitly attaches IP destination address and port # to each packet
- receiver extracts sender IP address and port# from received packet

UDP: transmitted data may be lost or received out-of-order

Application viewpoint:

- UDP provides *unreliable* transfer of groups of bytes (“datagrams”) between client and server processes

Client/server socket interaction: UDP



server (running on serverIP)

client



create socket:

```
clientSocket =  
socket(AF_INET, SOCK_DGRAM)
```

Client/server socket interaction: UDP



server (running on serverIP)

client



create socket:

```
clientSocket =  
socket(AF_INET, SOCK_DGRAM)
```

Client/server socket interaction: UDP



server (running on serverIP)

client



create socket:

```
clientSocket =  
socket(AF_INET, SOCK_DGRAM)
```

*Bind method: request a specific
port number for the socket*

Client/server socket interaction: UDP



server (running on serverIP)

```
create socket, port= x:  
serverSocket =  
socket(AF_INET, SOCK_DGRAM)
```

client



create socket:

```
clientSocket =  
socket(AF_INET, SOCK_DGRAM)
```

Client/server socket interaction: UDP



server (running on serverIP)

```
create socket, port= x:  
serverSocket =  
socket(AF_INET,SOCK_DGRAM)
```

client



```
create socket:  
clientSocket =  
socket(AF_INET,SOCK_DGRAM)
```

↓
Create datagram with serverIP address
And port=x; send datagram via
clientSocket

Client/server socket interaction: UDP



server (running on serverIP)

create socket, port= x:
`serverSocket =
socket(AF_INET,SOCK_DGRAM)`

read datagram from
`serverSocket`

client



create socket:
`clientSocket =
socket(AF_INET,SOCK_DGRAM)`

Create datagram with serverIP address
And port=x; send datagram via
`clientSocket`



Client/server socket interaction: UDP



server (running on serverIP)

create socket, port= x:
`serverSocket =
socket(AF_INET,SOCK_DGRAM)`

read datagram from
`serverSocket`

write reply to
`serverSocket`
specifying
client address,
port number

client



create socket:
`clientSocket =
socket(AF_INET,SOCK_DGRAM)`

Create datagram with serverIP address
And port=x; send datagram via
`clientSocket`

Client/server socket interaction: UDP



server (running on serverIP)

create socket, port= x:
`serverSocket =
socket(AF_INET,SOCK_DGRAM)`

read datagram from
`serverSocket`

write reply to
`serverSocket`
specifying
client address,
port number

client



create socket:
`clientSocket =
socket(AF_INET,SOCK_DGRAM)`

Create datagram with serverIP address
And port=x; send datagram via
`clientSocket`

read datagram from
`clientSocket`

close
`clientSocket`

Example app: UDP client

Python UDPClient

```
from socket import *
serverName = 'hostname'
serverPort = 12000
clientSocket = socket(AF_INET,
                      SOCK_DGRAM)
message = input('Input lowercase sentence:')
clientSocket.sendto(message.encode(),
                    (serverName, serverPort))
modifiedMessage, serverAddress =
    clientSocket.recvfrom(2048)
print(modifiedMessage.decode())
clientSocket.close()
```

Example app: UDP client

Python UDPClient

include Python's socket library →

```
from socket import *
serverName = 'hostname'
serverPort = 12000
clientSocket = socket(AF_INET,
                      SOCK_DGRAM)
message = input('Input lowercase sentence:')
clientSocket.sendto(message.encode(),
                    (serverName, serverPort))
modifiedMessage, serverAddress =
    clientSocket.recvfrom(2048)
print(modifiedMessage.decode())
clientSocket.close()
```

Example app: UDP client

Python UDPClient

```
from socket import *
serverName = 'hostname'
serverPort = 12000
create UDP socket → clientSocket = socket(AF_INET,
                                           SOCK_DGRAM)
message = input('Input lowercase sentence:')
clientSocket.sendto(message.encode(),
                    (serverName, serverPort))
modifiedMessage, serverAddress =
    clientSocket.recvfrom(2048)
print(modifiedMessage.decode())
clientSocket.close()
```

Example app: UDP client

Python UDPClient

```
from socket import *
serverName = 'hostname'
serverPort = 12000
clientSocket = socket(AF_INET,
                      SOCK_DGRAM)
get user keyboard input → message = input('Input lowercase sentence:')
clientSocket.sendto(message.encode(),
                    (serverName, serverPort))
modifiedMessage, serverAddress =
    clientSocket.recvfrom(2048)
print(modifiedMessage.decode())
clientSocket.close()
```

Example app: UDP client

Python UDPClient

```
from socket import *
serverName = 'hostname'
serverPort = 12000
clientSocket = socket(AF_INET,
                      SOCK_DGRAM)
message = input('Input lowercase sentence:')
attach server name, port to message; send into socket → clientSocket.sendto(message.encode(),
                                                                    (serverName, serverPort))
modifiedMessage, serverAddress =
    clientSocket.recvfrom(2048)
print(modifiedMessage.decode())
clientSocket.close()
```

Example app: UDP client

Python UDPClient

```
from socket import *
serverName = 'hostname'
serverPort = 12000
clientSocket = socket(AF_INET,
                      SOCK_DGRAM)
message = input('Input lowercase sentence:')
clientSocket.sendto(message.encode(),
                    (serverName, serverPort))
read reply data (bytes) from socket → modifiedMessage, serverAddress =
                                         clientSocket.recvfrom(2048)
print(modifiedMessage.decode())
clientSocket.close()
```

Example app: UDP client

Python UDPClient

```
from socket import *
serverName = 'hostname'
serverPort = 12000
clientSocket = socket(AF_INET,
                      SOCK_DGRAM)
message = input('Input lowercase sentence:')
clientSocket.sendto(message.encode(),
                    (serverName, serverPort))
modifiedMessage, serverAddress =
    clientSocket.recvfrom(2048)
print(modifiedMessage.decode())
clientSocket.close()
```

print out received string and close socket →

Example app: UDP server

Python UDPServer



```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)
serverSocket.bind(('', serverPort))
print('The server is ready to receive')
while True:
    message, clientAddress = serverSocket.recvfrom(2048)
    modifiedMessage = message.decode().upper()
    serverSocket.sendto(modifiedMessage.encode(),
                        clientAddress)
```

Example app: UDP server

Python UDPServer

```
from socket import *  
serverPort = 12000  
create UDP socket → serverSocket = socket(AF_INET, SOCK_DGRAM)  
serverSocket.bind(('', serverPort))  
print('The server is ready to receive')  
while True:  
    message, clientAddress = serverSocket.recvfrom(2048)  
    modifiedMessage = message.decode().upper()  
    serverSocket.sendto(modifiedMessage.encode(),  
                        clientAddress)
```

Example app: UDP server

Python UDPServer

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)
bind socket to local port number 12000 → serverSocket.bind(('', serverPort))
print('The server is ready to receive')
while True:
    message, clientAddress = serverSocket.recvfrom(2048)
    modifiedMessage = message.decode().upper()
    serverSocket.sendto(modifiedMessage.encode(),
                        clientAddress)
```

Example app: UDP server

Python UDPServer

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)
serverSocket.bind(('', serverPort))
print('The server is ready to receive')

loop forever → while True:
    message, clientAddress = serverSocket.recvfrom(2048)
    modifiedMessage = message.decode().upper()
    serverSocket.sendto(modifiedMessage.encode(),
                        clientAddress)
```

Example app: UDP server

Python UDPServer

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)
serverSocket.bind(('', serverPort))
print('The server is ready to receive')
while True:
    message, clientAddress = serverSocket.recvfrom(2048)
    modifiedMessage = message.decode().upper()
    serverSocket.sendto(modifiedMessage.encode(),
                        clientAddress)
```

Read from UDP socket into message, getting
client's address (client IP and port) →

Example app: UDP server

Python UDPServer

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)
serverSocket.bind(('', serverPort))
print('The server is ready to receive')
while True:
    message, clientAddress = serverSocket.recvfrom(2048)
    modifiedMessage = message.decode().upper()
    serverSocket.sendto(modifiedMessage.encode(),
                        clientAddress)
```

send upper case string back to this client →

Socket programming with TCP

Client must contact server

- server process must first be running
- server must have created socket (door) that welcomes client's contact

Socket programming with TCP

Client must contact server

- server process must first be running
- server must have created socket (door) that welcomes client's contact

Client contacts server by:

- Creating TCP socket, specifying IP address, port number of server process
- *when client creates socket:* client TCP establishes connection to server TCP

Socket programming with TCP

Client must contact server

- server process must first be running
- server must have created socket (door) that welcomes client's contact (*welcoming socket*)

Client contacts server by:

- Creating TCP socket, specifying IP address, port number of server process
- *when client creates socket*: client TCP establishes connection to server TCP

- when contacted by client, *server TCP creates new socket* for server process to communicate with that particular client
 - allows server to talk with multiple clients
 - client source port # and IP address used to distinguish clients (more in Chap 3)

Socket programming with TCP

Client must contact server

- server process must first be running
- server must have created socket (door) that welcomes client's contact

Client contacts server by:

- Creating TCP socket, specifying IP address, port number of server process
- *when client creates socket:* client TCP establishes connection to server TCP

- when contacted by client, *server TCP creates new socket* for server process to communicate with that particular client
 - allows server to talk with multiple clients
 - client source port # and IP address used to distinguish clients (more in Chap 3)
- welcoming socket different from new socket
 - Same port number

Socket programming with TCP

Client must contact server

- server process must first be running
- server must have created socket (door) that welcomes client's contact

Client contacts server by:

- Creating TCP socket, specifying IP address, port number of server process
- *when client creates socket*: client TCP establishes connection to server TCP

- when contacted by client, *server TCP creates new socket* for server process to communicate with that particular client
 - allows server to talk with multiple clients
 - client source port # and IP address used to distinguish clients (more in Chap 3)

Application viewpoint

TCP provides reliable, in-order byte-stream transfer ("pipe") between client and server processes

Client/server socket interaction: TCP



server (running on hostid)

client



Reminder:

- TCP is a connection-oriented service
- need handshake between client and server to send application level messages
- Client TCP socket
- Server side TCP socket
- Server needs to be up and running waiting for a client's connection request

Client/server socket interaction: TCP



server (running on hostid)

client



welcoming socket,
or listening socket

create socket,
port=**x**, for incoming
request:
serverSocket = socket()



Client/server socket interaction: TCP



server (running on hostid)

client



create socket,
port=x, for incoming
request:

`serverSocket = socket()`



wait for incoming
connection request

`connectionSocket =
serverSocket.accept()`

Client/server socket interaction: TCP



server (running on `hostid`)

create socket,
port=`x`, for incoming
request:

`serverSocket = socket()`



wait for incoming
connection request

`connectionSocket =
serverSocket.accept()`

client



create socket,
connect to **`hostid`**, port=`x`
`clientSocket = socket()`

Client/server socket interaction: TCP



server (running on `hostid`)

client



create socket,
port=`x`, for incoming
request:

`serverSocket = socket()`



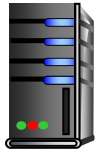
wait for incoming
connection request

`connectionSocket =`
`serverSocket.accept()`

← — — — TCP — — — →
connection setup

create socket,
connect to `hostid`, port=`x`
`clientSocket = socket()`

Client/server socket interaction: TCP



server (running on `hostid`)

client



create socket,
port=`x`, for incoming
request:

`serverSocket = socket()`



wait for incoming
connection request

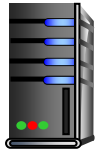
`connectionSocket =`
`serverSocket.accept()`

← — — — TCP — — — →
connection setup

create socket,
connect to **`hostid`**, port=`x`
`clientSocket = socket()`

- New socket: connection socket
 - (socket that the server side application will use to communicate with the client side application)

Client/server socket interaction: TCP



server (running on `hostid`)

client



create socket,
port=`x`, for incoming
request:

`serverSocket = socket()`



wait for incoming
connection request

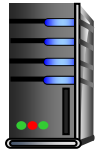
`connectionSocket =`
`serverSocket.accept()`

TCP
connection setup

create socket,
connect to **`hostid`**, port=`x`
`clientSocket = socket()`

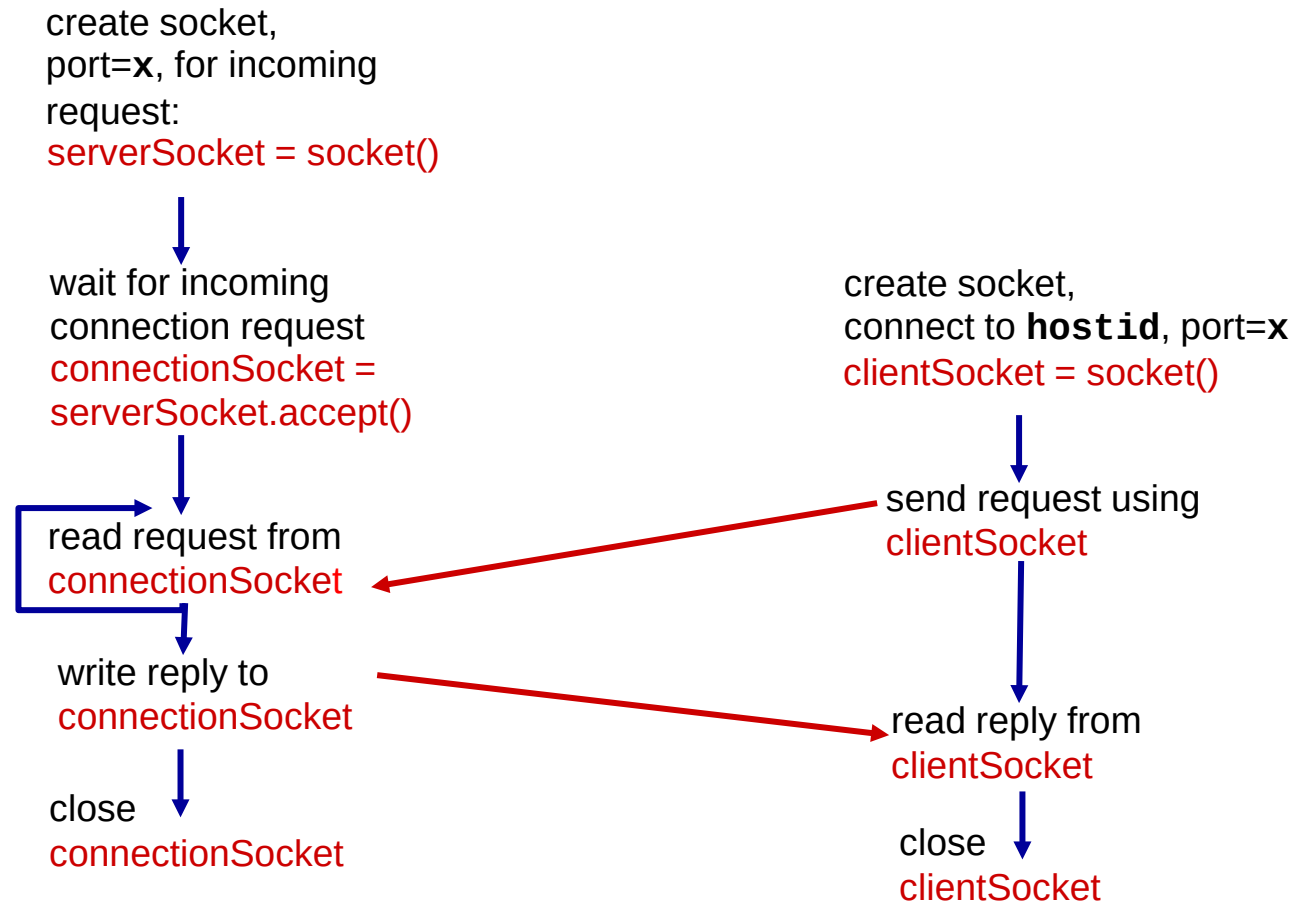
- New socket: connection socket
 - (socket that the server side application will use to communicate with the client side application)
- TCP level message to the client
 - (let the TCP client know that the connection has been established)

Client/server socket interaction: TCP



server (running on `hostid`)

client



Example app: TCP client

Python TCPClient



```
from socket import *
serverName = 'servername'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName,serverPort))
sentence = input('Input lowercase sentence:')
clientSocket.send(sentence.encode())
modifiedSentence = clientSocket.recv(1024)
print ('From Server:', modifiedSentence.decode())
clientSocket.close()
```

Example app: TCP client

Python TCPClient

create TCP socket for server,
remote port 12000



```
from socket import *
serverName = 'servername'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName, serverPort))
sentence = input('Input lowercase sentence:')
clientSocket.send(sentence.encode())
modifiedSentence = clientSocket.recv(1024)
print ('From Server:', modifiedSentence.decode())
clientSocket.close()
```

Example app: TCP client

Python TCPClient

Connect to server



```
from socket import *
serverName = 'servername'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect(serverName,serverPort)
sentence = input('Input lowercase sentence:')
clientSocket.send(sentence.encode())
modifiedSentence = clientSocket.recv(1024)
print ('From Server:', modifiedSentence.decode())
clientSocket.close()
```

Example app: TCP client

Python TCPClient

Read input from keyboard

Send input

```
from socket import *
serverName = 'servername'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName,serverPort))
sentence = input('Input lowercase sentence:')
clientSocket.send(sentence.encode())
modifiedSentence = clientSocket.recv(1024)
print ('From Server:', modifiedSentence.decode())
clientSocket.close()
```

Example app: TCP client

Python TCPClient

```
from socket import *
serverName = 'servername'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName,serverPort))
sentence = input('Input lowercase sentence:')
clientSocket.send(sentence.encode())
modifiedSentence = clientSocket.recv(1024)
print ('From Server:', modifiedSentence.decode())
clientSocket.close()
```

No need to attach server name, port



Example app: TCP server

Python TCP Server

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET,SOCK_STREAM)
serverSocket.bind(('',serverPort))
serverSocket.listen(1)
print('The server is ready to receive')
while True:
    connectionSocket, addr = serverSocket.accept()

    sentence = connectionSocket.recv(1024).decode()
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence.
                           encode())

    connectionSocket.close()
```

Example app: TCP server

Python TCP Server

create TCP welcoming socket



```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET,SOCK_STREAM)
serverSocket.bind(('',serverPort))
serverSocket.listen(1)
print('The server is ready to receive')
while True:
    connectionSocket, addr = serverSocket.accept()

    sentence = connectionSocket.recv(1024).decode()
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence.
                           encode())

    connectionSocket.close()
```

Example app: TCP server

Python TCPServer

server begins listening for
incoming TCP requests



```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET,SOCK_STREAM)
serverSocket.bind(('',serverPort))
serverSocket.listen(1)
print('The server is ready to receive')
while True:
    connectionSocket, addr = serverSocket.accept()

    sentence = connectionSocket.recv(1024).decode()
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence.
                           encode())

    connectionSocket.close()
```

Example app: TCP server

Python TCPServer

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET,SOCK_STREAM)
serverSocket.bind(('',serverPort))
serverSocket.listen(1)
print('The server is ready to receive')
loop forever → while True:
    connectionSocket, addr = serverSocket.accept()

    sentence = connectionSocket.recv(1024).decode()
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence.
                           encode())

    connectionSocket.close()
```

Example app: TCP server

Python TCPServer

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_STREAM)
serverSocket.bind(('', serverPort))
serverSocket.listen(1)
print('The server is ready to receive')
while True:
```

server waits on accept() for incoming requests, new socket created on return



```
    connectionSocket, addr = serverSocket.accept()
```

```
        sentence = connectionSocket.recv(1024).decode()
        capitalizedSentence = sentence.upper()
        connectionSocket.send(capitalizedSentence.
                                encode())
        connectionSocket.close()
```

Example app: TCP server

Python TCP Server

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_STREAM)
serverSocket.bind(('', serverPort))
serverSocket.listen(1)
print('The server is ready to receive')
while True:
    connectionSocket, addr = serverSocket.accept()
    sentence = connectionSocket.recv(1024).decode()
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence.
                           encode())
    connectionSocket.close()
```

read bytes from socket (but
not address as in UDP) →

Example app: TCP server

Python TCP Server

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_STREAM)
serverSocket.bind(('', serverPort))
serverSocket.listen(1)
print('The server is ready to receive')
while True:
    connectionSocket, addr = serverSocket.accept()

    sentence = connectionSocket.recv(1024).decode()
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence.
                           encode())

    connectionSocket.close()
```

close connection to this client (but *not* →
welcoming socket)

Socket programming – recap key points

Typical network application

- client program and a server program are executed
- client process and server process are created
- process communicate through sockets

Socket programming – recap key points

Typical network application

- client program and a server program are executed
- client process and server process are created
- process communicate through sockets
- a developer writes code for client and server programs
 - With TCP
 - With UDP

Socket programming - recap key points

Typical network application

- client program and a server program are executed
- client process and server process are created
- process communicate through sockets
- a developer writes code for client and server programs
 - With TCP
 - Connection oriented and reliable
 - With UDP
 - Connectionless, no guarantee of packets delivery

Socket programming - recap key points

Typical network application

- Client initiates contact with the server
- Server has to be ready, running as a process

Socket programming - recap key points

UDP sockets

- Client socket
- Server socket

TCP sockets

- Client socket at client
- Welcoming socket at server
- Connection socket at server

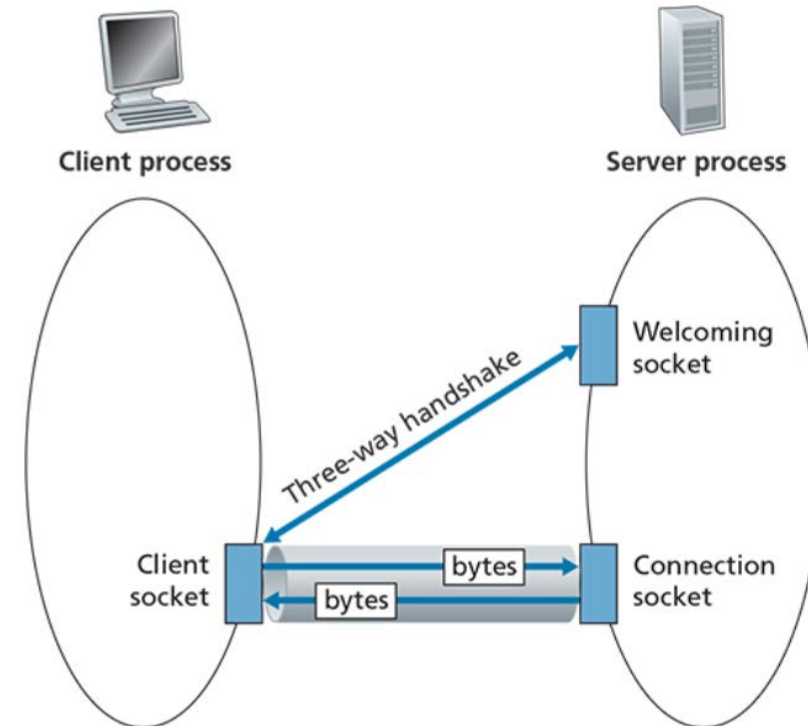
Socket programming - recap key points

UDP sockets

- Client socket
- Server socket

TCP sockets

- Client socket at client
- Welcoming socket at server
- Connection socket at server



That's all for now

- Principles of network applications
- Web and HTTP
- E-mail, SMTP, IMAP
- The Domain Name System DNS
- P2P applications
- video streaming and content distribution networks
- **socket programming with UDP and TCP**

